Individual Assignment I: Machine Learning Foundation

Data Preparation — UCI Bank Marketing Dataset

GitHub Repository: https://github.com/christophrrrrr/ML-fundamentals-2026

This notebook implements disciplined data preparation and feature engineering for the UCI Bank Marketing Dataset (`bank-additional.csv`). The emphasis is on **correct pipeline ordering**, **principled justification of every decision**, and **rigorous avoidance of data leakage** — not on maximising model performance.

```
import os
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.patches as mpatches
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score,
    ConfusionMatrixDisplay, classification_report, f1_score,
    precision_recall_curve, average_precision_score
)
from sklearn.feature_selection import VarianceThreshold
from sklearn.pipeline import Pipeline as SklearnPipeline
from imblearn.pipeline import Pipeline
from imblearn.over_sampling import RandomOverSampler
import warnings
warnings.filterwarnings('ignore', category=UserWarning)
warnings.filterwarnings('ignore', category=FutureWarning)

# Consistent plot style
plt.rcParams['figure.dpi'] = 100
plt.rcParams['axes.spines.top'] = False
plt.rcParams['axes.spines.right'] = False
print('Libraries loaded successfully.')
```

```
Libraries loaded successfully.
```

Task 1: Identifying the Prediction Target

Lecture material: Lecture 1 (Problem Formulation), Lecture 2 (Data Inspection).

Target Variable

The target variable is `y` — a binary indicator (`yes`/`no`) of whether a client subscribed to a term deposit following a telemarketing contact. This directly corresponds to the stated business objective: predicting whether a client will convert, given information available at the time the call is placed.

Variables That Must NOT Be Treated as Targets

1. **`duration` (dropped entirely — data leakage):** Call duration is only known *after* the call ends, making it unavailable at genuine prediction time. More critically, it is a near-perfect proxy for the outcome: clients who subscribe stay on the call substantially longer. Including `duration` as a feature — or mistaking it for a target — would constitute **direct target leakage**. The UCI dataset documentation explicitly warns against using it for realistic predictive models. It is removed before any other processing.
2. **`nr.employed` (could superficially resemble an output metric):** The number of bank employees is a macroeconomic indicator that tracks economic cycles and can appear to be an aggregate outcome of the campaigns. In reality it is an exogenous input measured at contact time — it does not depend on any individual call outcome and is legitimately available as a feature. However, a student unfamiliar with the dataset could mistake it for a target measuring campaign scale.
3. **`previous` (could be mistaken for a campaign outcome):** `previous` records the number of contacts made *before* the current campaign. A superficial reading might suggest this is an outcome variable — "how many times did we have to contact this person?" — when in fact it is historical input data describing the client's prior engagement history, available at prediction time.

Task 2: Data Loading and Exploration

Lecture material: Lecture 1 (Problem Formulation), Lecture 2 (Data Inspection and EDA).

`bank-additional.csv` is the 10% stratified sample (4,119 rows) of the full dataset. The UCI format uses semicolon separators.

```
full_filepath   = 'data/bank-additional-full.csv'
sample_filepath = 'data/bank-additional.csv'
github_url      = 'https://raw.githubusercontent.com/christophrrrrr/ML-fundamentals-2026/main/data/bank-additional.csv'

if os.path.exists(full_filepath):
    print(f'Loading full dataset from: {full_filepath}')
    df = pd.read_csv(full_filepath, sep=';')
elif os.path.exists(sample_filepath):
    print(f'Loading 10% sample from: {sample_filepath}')
    df = pd.read_csv(sample_filepath, sep=';')
else:
    print('Local file not found — downloading 10% sample from GitHub...')
    df = pd.read_csv(github_url, sep=';')

print(f'\nDataset shape: {df.shape[0]} rows × {df.shape[1]} columns')
print('\n--- Data Types ---')
print(df.dtypes)
print('\n--- Summary Statistics (numerical) ---')
display(df.describe())
```

```
Local file not found — downloading 10% sample from GitHub...

Dataset shape: 4119 rows × 21 columns

--- Data Types ---
age                 int64
job                object
marital            object
education          object
default            object
housing            object
loan               object
contact            object
month              object
day_of_week        object
duration            int64
campaign            int64
pdays               int64
previous            int64
poutcome           object
emp.var.rate      float64
cons.price.idx    float64
cons.conf.idx     float64
euribor3m         float64
nr.employed       float64
y                  object
dtype: object

--- Summary Statistics (numerical) ---
```

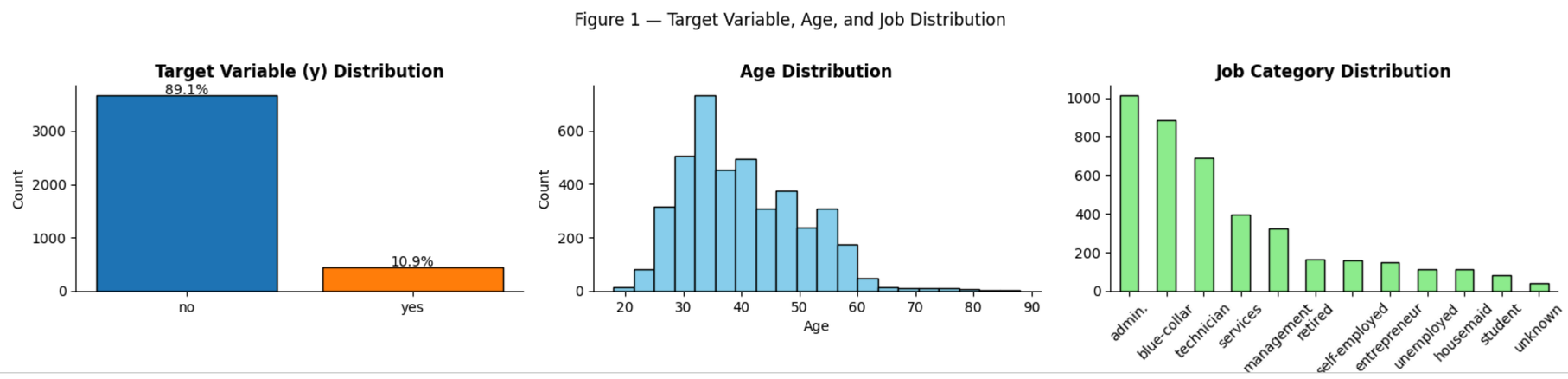
	age	duration	campaign	pdays	previous	emp.var.rate	cons.price.idx	cons.conf.idx	euribor3m	nr.employed
count	4119.000000	4119.000000	4119.000000	4119.000000	4119.000000	4119.000000	4119.000000	4119.000000	4119.000000	4119.000000
mean	40.113620	256.788055	2.537266	960.422190	0.190337	0.084972	93.579704	-40.499102	3.621356	5166.481695
std	10.313362	254.703736	2.568159	191.922786	0.541788	1.563114	0.579349	4.594578	1.733591	73.667904
min	18.000000	0.000000	1.000000	0.000000	0.000000	-3.400000	92.201000	-50.800000	0.635000	4963.600000
25%	32.000000	103.000000	1.000000	999.000000	0.000000	-1.800000	93.075000	-42.700000	1.334000	5099.100000
50%	38.000000	181.000000	2.000000	999.000000	0.000000	1.100000	93.749000	-41.800000	4.857000	5191.000000
75%	47.000000	317.000000	3.000000	999.000000	0.000000	1.400000	93.994000	-36.400000	4.961000	5228.100000
max	88.000000	3643.000000	35.000000	999.000000	6.000000	1.400000	94.767000	-26.900000	5.045000	5228.100000

Variable Classification

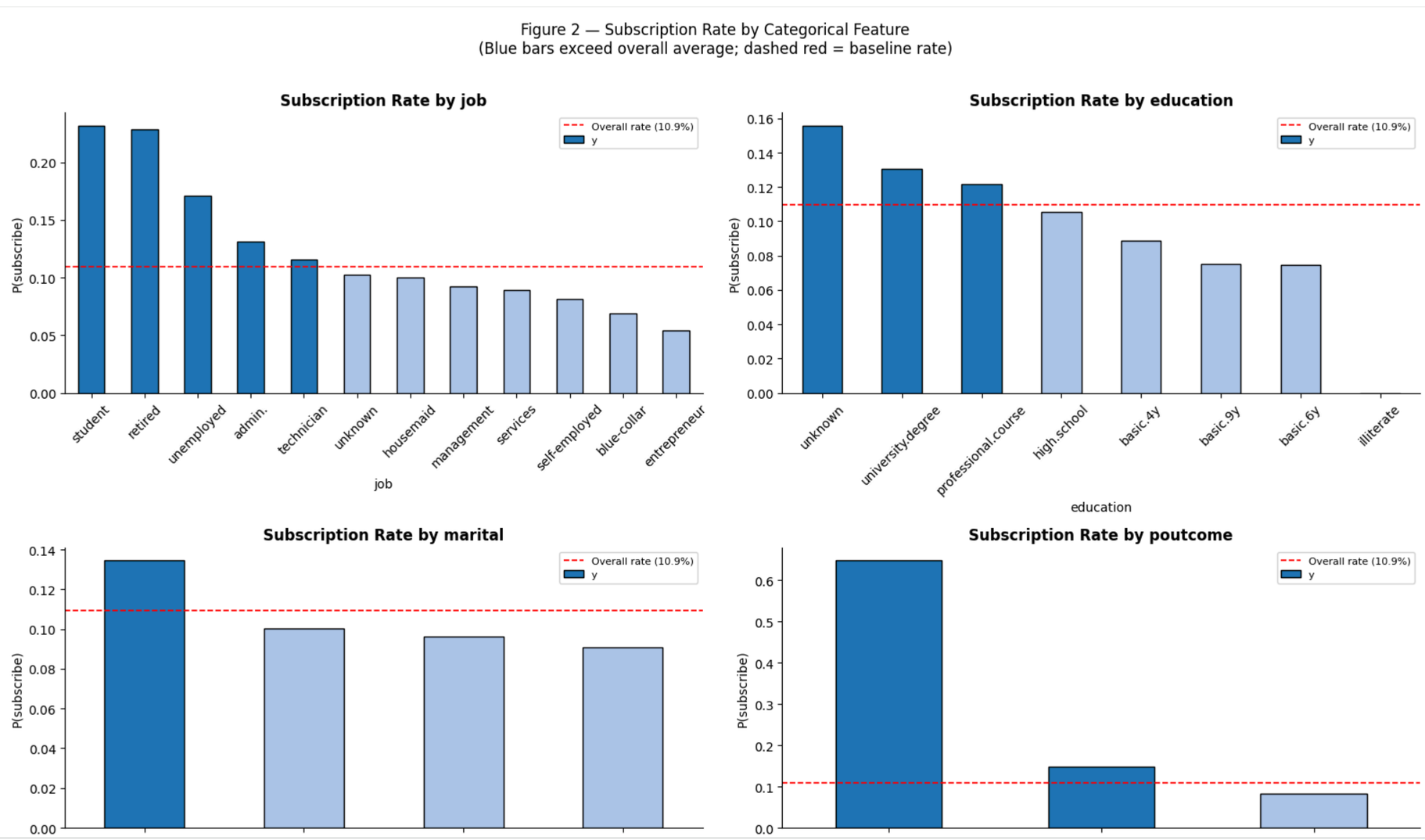
Numerical (10): `age`, `duration`*, `campaign`, `pdays`, `previous`, `emp.var.rate`, `cons.price.idx`, `cons.conf.idx`, `euribor3m`, `nr.employed`
Categorical (11): `job`, `marital`, `education`, `default`, `housing`, `loan`, `contact`, `month`, `day_of_week`, `poutcome`, `y`

* `duration` is identified here for completeness but dropped immediately. `pdays` will be decomposed into a binary indicator (`prev_contacted`) because 96% of its values are the sentinel `999`.

```
# ── Figure 1: Target distribution and class imbalance ──────────────────
y_counts = df['y'].value_counts()
y_pct    = df['y'].value_counts(normalize=True)

fig, axes = plt.subplots(1, 3, figsize=(16, 4))

# Target variable
bars = axes[0].bar(y_counts.index, y_counts.values,
                   color=['#1f77b4', '#ff7f0e'], edgecolor='black')
axes[0].set_title('Target Variable (y) Distribution', fontweight='bold')
axes[0].set_ylabel('Count')
for bar, pct in zip(bars, y_pct.values):
    axes[0].text(bar.get_x() + bar.get_width()/2, bar.get_height() + 20,
                 f'{pct:.1%}', ha='center', fontsize=10)

# Age — hypothesis: older clients may have more disposable savings → higher subscription
axes[1].hist(df['age'], bins=20, color='skyblue', edgecolor='black')
axes[1].set_title('Age Distribution', fontweight='bold')
axes[1].set_xlabel('Age')
axes[1].set_ylabel('Count')

# Job — hypothesis: retired / student / management profiles may behave differently
df['job'].value_counts().plot(kind='bar', color='lightgreen', edgecolor='black', ax=axes[2])
axes[2].set_title('Job Category Distribution', fontweight='bold')
axes[2].tick_params(axis='x', rotation=45)

plt.suptitle('Figure 1 — Target Variable, Age, and Job Distribution', y=1.01, fontsize=12)
plt.tight_layout()
plt.show()

print(f'Class distribution: no={y_counts["no"]} ({y_pct["no"]:.1%}),  yes={y_counts["yes"]} ({y_pct["yes"]:.1%})')
```

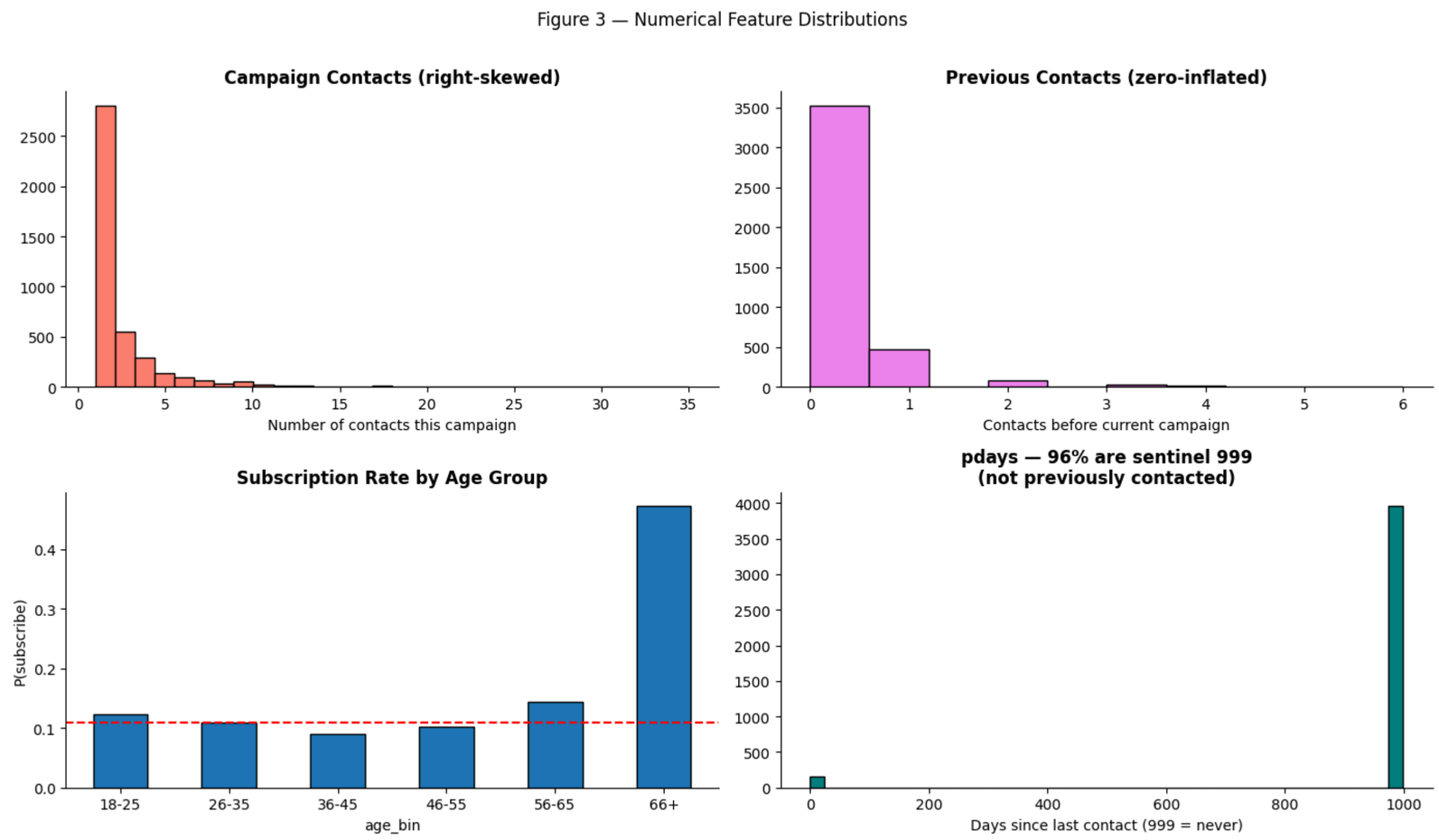



```
# ── Figure 2: Subscription rate by key categorical features ──────────────
# This goes beyond marginal distributions — it directly asks which categories
# are predictive of subscription, informing encoding and feature selection.

cat_explore = ['job', 'education', 'marital', 'poutcome']
fig, axes = plt.subplots(2, 2, figsize=(16, 10))
axes = axes.flatten()

for i, col in enumerate(cat_explore):
    sub_rate = df.groupby(col)['y'].apply(lambda x: (x == 'yes').mean()).sort_values(ascending=False)
    colors = ['#1f77b4' if v > df['y'].eq('yes').mean() else '#aec6e8' for v in sub_rate.values]
    sub_rate.plot(kind='bar', ax=axes[i], color=colors, edgecolor='black')
    axes[i].axhline(df['y'].eq('yes').mean(), color='red', linestyle='--', linewidth=1.2,
                    label=f'Overall rate ({df["y"].eq("yes").mean():.1%})')
    axes[i].set_title(f'Subscription Rate by {col}', fontweight='bold')
    axes[i].set_ylabel('P(subscribe)')
    axes[i].tick_params(axis='x', rotation=45)
    axes[i].legend(fontsize=8)

plt.suptitle('Figure 2 — Subscription Rate by Categorical Feature\n'
             '(Blue bars exceed overall average; dashed red = baseline rate)',
             fontsize=12, y=1.01)
plt.tight_layout()
plt.show()

print('Observation: poutcome=success strongly predicts subscription (~65% rate vs 11% overall).')
print('Observation: retired and student job categories subscribe above the average rate.')
print('Observation: marital status shows modest differentiation — single clients subscribe slightly more.')
```



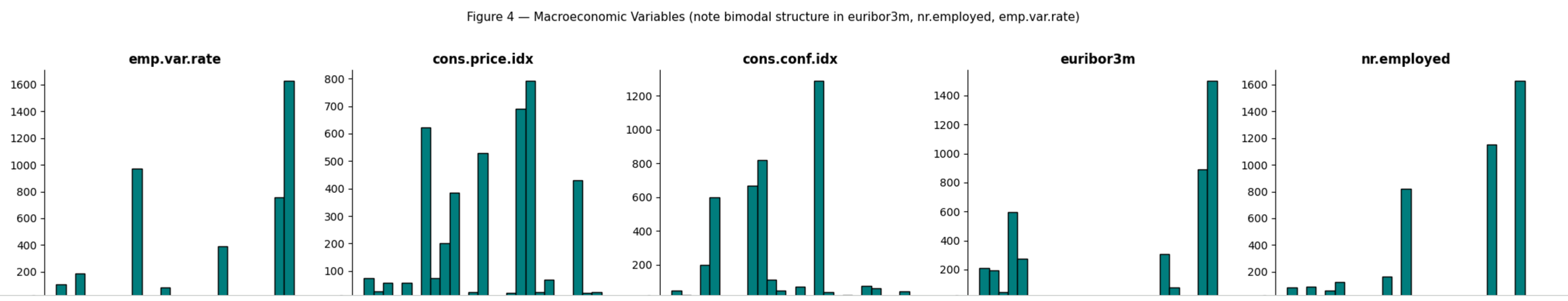

```
# ── Figure 3: Numerical feature distributions ────────────────────────────
fig2, axes2 = plt.subplots(2, 2, figsize=(14, 8))

# campaign — right-skewed: most clients contacted 1-3 times, long tail to 35
axes2[0,0].hist(df['campaign'], bins=30, color='salmon', edgecolor='black')
axes2[0,0].set_title('Campaign Contacts (right-skewed)', fontweight='bold')
axes2[0,0].set_xlabel('Number of contacts this campaign')

# previous — zero-inflated: vast majority of clients have no prior contact
axes2[0,1].hist(df['previous'], bins=10, color='violet', edgecolor='black')
axes2[0,1].set_title('Previous Contacts (zero-inflated)', fontweight='bold')
axes2[0,1].set_xlabel('Contacts before current campaign')

# age vs subscription rate (binned) — hypothesis check
df['age_bin'] = pd.cut(df['age'], bins=[17,25,35,45,55,65,100],
                       labels=['18-25','26-35','36-45','46-55','56-65','66+'])
age_sub = df.groupby('age_bin', observed=True)['y'].apply(lambda x: (x=='yes').mean())
age_sub.plot(kind='bar', ax=axes2[1,0], color='#1f77b4', edgecolor='black')
axes2[1,0].axhline(df['y'].eq('yes').mean(), color='red', linestyle='--')
axes2[1,0].set_title('Subscription Rate by Age Group', fontweight='bold')
axes2[1,0].set_ylabel('P(subscribe)')
axes2[1,0].tick_params(axis='x', rotation=0)

# pdays — sentinel distribution
axes2[1,1].hist(df['pdays'], bins=40, color='teal', edgecolor='black')
axes2[1,1].set_title('pdays — 96% are sentinel 999\n(not previously contacted)', fontweight='bold')
axes2[1,1].set_xlabel('Days since last contact (999 = never)')

plt.suptitle('Figure 3 — Numerical Feature Distributions', fontsize=12, y=1.01)
plt.tight_layout()
plt.show()
```

```
df = df.drop(columns=['age_bin'])  # clean up temporary column
print('Observation: campaign is right-skewed — a log transform could help, but Logistic Regression')
print('  is not strictly sensitive to feature skewness after standardisation.')
print('Observation: older clients (56+) subscribe at above-average rates, confirming age carries signal.')
print('Observation: pdays=999 acts as a sentinel; a binary prev_contacted flag captures the useful information.')
```




```
Observation: campaign is right-skewed — a log transform could help, but Logistic Regression
  is not strictly sensitive to feature skewness after standardisation.
Observation: older clients (56+) subscribe at above-average rates, confirming age carries signal.
```

```
# ── Figure 4: Macroeconomic variables — bimodality check ──────────────
macro_vars = ['emp.var.rate', 'cons.price.idx', 'cons.conf.idx', 'euribor3m', 'nr.employed']
fig3, axes3 = plt.subplots(1, 5, figsize=(20, 4))
for i, var in enumerate(macro_vars):
    axes3[i].hist(df[var], bins=25, color='teal', edgecolor='black')
    axes3[i].set_title(var, fontweight='bold')
plt.suptitle('Figure 4 — Macroeconomic Variables (note bimodal structure in euribor3m, nr.employed, emp.var.rate)',
             fontsize=11, y=1.02)
plt.tight_layout()
plt.show()

print('Observation: euribor3m, nr.employed, and emp.var.rate all show bimodal distributions clustering')
print('  into two economic regimes. This suggests high inter-variable correlation — to be quantified')
print('  in Feature Selection (Section 9).')
print('Observation: cons.price.idx shows a roughly unimodal distribution — it may capture a distinct')
print('  macroeconomic dimension not fully absorbed by euribor3m.')
```




```
# ── Figure 5: default column — distribution of non-missing values ──────
# This is important: 19.5% of 'default' is unknown (implicit missing).
# Inspecting the non-missing distribution informs imputation strategy.
default_counts = df['default'].value_counts(dropna=False)
print('--- default column value counts (including NaN/unknown before cleaning) ---')
print(default_counts)
print(f'\nNon-missing breakdown:')
non_missing = df[df['default'] != 'unknown']['default'].value_counts()
print(non_missing)
print(f'\nAmong clients with known default status:')
print(f'  "no" : {non_missing.get("no", 0)} ({non_missing.get("no", 0)/non_missing.sum():.1%})')
print(f'  "yes": {non_missing.get("yes", 0)} ({non_missing.get("yes", 0)/non_missing.sum():.1%})')
print()
print('Key finding: default="yes" is extremely rare (<1% of non-missing values).')
print('Mode imputation would map everything to "no", effectively erasing the default signal entirely.')
print('Encoding "unknown" as a separate "missing" category is the correct strategy:')
print('  it preserves the pattern that missingness itself may be predictive.')
```

```
--- default column value counts (including NaN/unknown before cleaning) ---
default
no         3315
unknown     803
yes           1
Name: count, dtype: int64

Non-missing breakdown:
default
no     3315
yes       1
Name: count, dtype: int64

Among clients with known default status:
  "no" : 3315 (100.0%)
  "yes": 1 (0.0%)

Key finding: default="yes" is extremely rare (<1% of non-missing values).
Mode imputation would map everything to "no", effectively erasing the default signal entirely.
Encoding "unknown" as a separate "missing" category is the correct strategy:
  it preserves the pattern that missingness itself may be predictive.
```

EDA Summary

Observation	Implication
Target is 89%/11% imbalanced	Resampling required; accuracy is a misleading metric
`pdays=999` in 96% of rows	Extract binary `prev_contacted`; raw pdays value meaningless
`default` has 19.5% implicit missing	Mode imputation would destroy signal; encode as `'missing'` category
`euribor3m`, `emp.var.rate`, `nr.employed` are bimodal & move together	High inter-variable correlation; multicollinearity to address in Feature Selection
`poutcome=success` predicts subscription at ~65% vs 11% overall	Strong feature; `poutcome` categories need careful one-hot encoding
`campaign` is right-skewed (long tail to 35)	Standardisation will centre this; consider extreme outlier treatment
`default="yes"` is <1% of non-missing values	Even after imputation, feature has very low variance — to be checked in Feature Selection

---

Task 3: Task Ordering

Lecture material: Lecture 2 (Data Splitting and Leakage), Lecture 5 (Preprocessing), Lecture 9 (ML Pipeline).

The tasks in this assignment are listed alphabetically as a deliberate challenge. The correct execution order is determined by one governing principle: **any operation that learns parameters from data must be fitted exclusively on the training set**. Below is the justified pipeline sequence.

Step	Task	Information Allowed	Information Forbidden	Leakage Risk if Misordered
1	**Target Identification & Data Loading**	Raw column names, dataset schema	Any statistics, distributions	None — no fitting occurs
2	**Structural Sentinel Cleaning**	Column names, known sentinel values (`'unknown'`, `999`)	Any computed statistics (means, modes)	None — purely structural, no learning
3	**Data Splitting**	Full cleaned raw dataset	Nothing from test/val	All downstream leakage is prevented here
4	**Missing Value Imputation**	`X_train` statistics only (median, mode)	`X_val` / `X_test` statistics	Imputing before split uses test-set central tendency in imputed values
5	**Categorical Encoding**	Category levels from `X_train` only	Categories unique to val/test	Fitting encoder on full data leaks category frequency information
6	**Feature Scaling**	Mean & std from `X_train` only	`X_val` / `X_test` scale	Scaling before split uses test-set mean/variance — distributional leakage

Step	Task	Information Allowed	Information Forbidden	Leakage Risk if Misordered
7	**Feature Selection**	Variance & correlation from `X_train` only	Val/test feature distributions	Selecting features on full data uses test variance/covariance
8	**Class Imbalance Resampling**	Training labels & features only	Val/test observations	Resampling before split contaminates val/test with synthetic duplicates from training observations
9	**Model Training & Evaluation**	Fitted pipeline on train; predict on val/test	Test set during model selection	Evaluating on test set during tuning makes it a de-facto validation set

Two Concrete Incorrect Ordering Examples

Example 1 — Feature Scaling before Data Splitting:
If `StandardScaler` is fitted on the full dataset, the mean and standard deviation are computed using all 4,119 rows — including the 618 observations that will become the test set. The scaler's parameters therefore encode information about the test set distribution. When those test observations are later standardised, the resulting values reflect their position relative to a mean that was partly determined by themselves. This violates the requirement that the test set must be treated as completely unseen. In practice this inflates the apparent quality of normalisation and can cause subtle overfitting to the test distribution.

Example 2 — Resampling before Data Splitting:
If `RandomOverSampler` duplicates minority-class rows before splitting, the duplicated samples are drawn from the complete dataset. When the dataset is subsequently split, some original observations (and their duplicates) will appear in different partitions — for instance, an original row in the test set and its duplicate in the training set. The model is effectively trained on observations it will later be evaluated on. Evaluation metrics (precision, recall, F1) will be optimistically biased because the model has already "seen" the test examples during training.

Task 4: Managing Missing Values (Part 1 — Identification & Structural Cleaning)

Lecture material: Lecture 2 (Data Inspection), Lecture 5 (Preprocessing and Pipeline Discipline).

Two types of missing values must be distinguished:

1. **Explicit NaN:** Not present in this CSV — the dataset uses string sentinels instead.
2. **Implicit missing — categorical `'unknown'`:** Encoded as the string `'unknown'` across six categorical columns. These are converted to `NaN` before splitting. This is a purely structural operation that computes no statistics, so it is safe to perform before the train/test split.
3. **Implicit missing — numerical sentinel `pdays=999`:** The value `999` is a sentinel meaning "this client was never previously contacted". It is not a genuine measurement of days elapsed. The raw magnitude carries no useful information (median of non-sentinel values is ~7 days), so it is replaced with a binary `prev_contacted` flag (1 = was contacted before, 0 = was not).

Data cleaning vs. modelling decisions:
Replacing `'unknown'` with `NaN` and extracting the `prev_contacted` flag are *data cleaning* decisions — correcting encoding artefacts. Deciding *how to impute* the resulting `NaN` values (Section 6) is a *modelling* decision that must wait until after splitting.

```
# Step 1: Drop duration (leakage — must happen before any analysis)
if 'duration' in df.columns:
    df = df.drop(columns=['duration'])
    print('Dropped: duration (target leakage — unavailable at prediction time)')

# Step 2: Quantify implicit missing values before cleaning
missing_records = []
for col in df.drop(columns=['y']).select_dtypes(include=['object']).columns:
    unk = (df[col] == 'unknown').sum()
    if unk > 0:
        missing_records.append({'Variable': col,
                                'Implicit Missing (unknown)': unk,
                                '% of Total': f'{unk/len(df)*100:.2f}%'})

pdays_sentinel = (df['pdays'] == 999).sum()
missing_records.append({'Variable': 'pdays',
                        'Implicit Missing (unknown)': pdays_sentinel,
                        '% of Total': f'{pdays_sentinel/len(df)*100:.2f}%'})

missing_df = pd.DataFrame(missing_records)
print('\n--- Implicit Missing Values Summary (before structural cleaning) ---')
display(missing_df)
```

```
Dropped: duration (target leakage — unavailable at prediction time)

--- Implicit Missing Values Summary (before structural cleaning) ---
```

	Variable	Implicit Missing (unknown)	% of Total
0	job	39	0.95%
1	marital	11	0.27%
2	education	167	4.05%
3	default	803	19.50%
4	housing	105	2.55%
5	loan	105	2.55%
6	pdays	3959	96.12%

Nächste Schritte: New interactive sheet

```
# Step 3: Structural cleaning — 'unknown' → NaN (no statistics computed)
for col in df.select_dtypes(include=['object']).columns:
    df[col] = df[col].replace('unknown', np.nan)

# Step 4: pdays sentinel → binary prev_contacted flag
# Rationale: 96.1% of pdays values are 999 (never contacted).
# The actual day-count for the ~4% who were contacted carries limited signal
# relative to the binary fact of prior contact (which, as the EDA showed,
# is associated with a 62% subscription rate vs 9% for non-contacted clients).
df['prev_contacted'] = (df['pdays'] != 999).astype(int)
df = df.drop(columns=['pdays'])

nan_after = df.isna().sum()
print('NaN counts after structural cleaning (these will be imputed post-split):')
print(nan_after[nan_after > 0].to_string())
print(f"\nprev_contacted: {(df['prev_contacted']==1).sum()} contacted ({(df['prev_contacted']==1).mean():.1%}), "
      f"{(df['prev_contacted']==0).sum()} not contacted ({(df['prev_contacted']==0).mean():.1%})")
```

```
NaN counts after structural cleaning (these will be imputed post-split):
job           39
marital       11
education    167
default      803
housing      105
loan         105

prev_contacted: 160 contacted (3.9%), 3959 not contacted (96.1%)
```

Task 5: Data Splitting

Lecture material: Lecture 2 (Data Splitting and Leakage), Lecture 9 (ML Pipeline).

The split must occur here — before any operation that learns from data. All subsequent steps (imputation, encoding, scaling, feature selection, resampling) will be fitted on `X_train` only.

Proportions: 70% train / 15% validation / 15% test
With 4,119 observations, a 70/15/15 split yields ~2,883 training, ~618 validation, and ~618 test samples. The training set must be large enough for stable parameter estimation (especially after resampling); 70% provides this while keeping two held-out sets for independent model selection (validation) and final generalisation estimation (test).

Why stratify on `y`?
With only ~11% positive examples, an unstratified random split has a non-trivial probability of producing a training set with significantly fewer positives (e.g., 8-9%), which would destabilise learning and make the resampling ratios unreliable. Stratification guarantees that all three sets preserve the original 89%/11% class ratio.

```
X = df.drop(columns=['y'])
y = df['y']

X_train, X_temp, y_train, y_temp = train_test_split(
    X, y, test_size=0.30, stratify=y, random_state=42
)
X_val, X_test, y_val, y_test = train_test_split(
    X_temp, y_temp, test_size=0.50, stratify=y_temp, random_state=42
)

print(f'X_train: {X_train.shape}  |  X_val: {X_val.shape}  |  X_test: {X_test.shape}')
print('\nStratification check — positive class rate per split:')
for name, ys in [('Full', y), ('Train', y_train), ('Val', y_val), ('Test', y_test)]:
    pct = (ys == 'yes').mean()
    print(f'  {name:6s}: {pct:.2%} positive')
print('\nAll splits preserve the original class distribution. ✓')
```

```
X_train: (2883, 19)  |  X_val: (618, 19)  |  X_test: (618, 19)

Stratification check — positive class rate per split:
  Full  : 10.95% positive
  Train : 10.96% positive
```

```
  Val   : 11.00% positive
  Test  : 10.84% positive

All splits preserve the original class distribution. ✓
```

Task 6: Managing Missing Values (Part 2 — Statistical Imputation)

Lecture material: Lecture 2 (Data Inspection), Lecture 5 (Preprocessing and Pipeline Discipline).

Imputation parameters (median, fill value) are computed from `X_train` only. This is enforced by embedding the imputer inside the `Pipeline` — `.fit()` is called only on training data, `.transform()` is applied to val/test without re-fitting.

Numerical imputation (median): No numerical NaNs exist post-cleaning, but median imputation is specified as a defensive measure. Median is preferred over mean for skewed distributions (`campaign`, `previous`).

Categorical imputation (constant `'missing'`): `NaN` values (converted from `'unknown'`) are replaced with the string `'missing'`. This is intentionally different from mode imputation for the following reason: the `'unknown'` entries are likely **not missing at random**. Banks may systematically omit credit default status for clients with complex profiles, and clients without declared housing loans may be more likely to be first-time contacts. Encoding missingness as its own category allows the model to learn a separate coefficient for the missing pattern — capturing a signal that mode imputation would destroy.

Special case — `default` (19.5% missing): As demonstrated in the EDA, `default='yes'` is <1% of non-missing values. Mode imputation would set all 803 unknown values to `'no'`, making the feature nearly constant and erasing any signal. The `'missing'` category preserves the pattern that the model can learn from.

```
num_cols = X_train.select_dtypes(include=[np.number]).columns.tolist()
cat_cols = X_train.select_dtypes(exclude=[np.number]).columns.tolist()

num_imputer = SimpleImputer(strategy='median')
cat_imputer = SimpleImputer(strategy='constant', fill_value='missing')

print(f'Numerical features ({len(num_cols)}): {num_cols}')
print(f'Categorical features ({len(cat_cols)}): {cat_cols}')
print('\nImputation strategy:')
print('  Numerical  → median (robust to skewness in campaign, previous)')
print('  Categorical → constant "missing" (preserves missingness pattern as informative category)')
```

```
Numerical features (9): ['age', 'campaign', 'previous', 'emp.var.rate', 'cons.price.idx', 'cons.conf.idx', 'euribor3m', 'nr.employed', 'prev_contacted']
Categorical features (10): ['job', 'marital', 'education', 'default', 'housing', 'loan', 'contact', 'month', 'day_of_week', 'poutcome']

Imputation strategy:
  Numerical  → median (robust to skewness in campaign, previous)
  Categorical → constant "missing" (preserves missingness pattern as informative category)
```

Task 7: Encoding Categorical Variables

Lecture material: Lecture 4 (Categorical Encoding), Lecture 6 (Linear Models), Lecture 9 (Feature Engineering).

Nominal vs Ordinal Classification

Variable	Type	Rationale
`job`	Nominal	No meaningful ordering between admin, technician, management, etc.
`marital`	Nominal	Single/married/divorced carry no ordinal progression
`contact`	Nominal	Telephone vs cellular — unordered
`month`	Nominal*	Cyclic structure exists but is not informative here (see below)
`day_of_week`	Nominal*	Same reasoning as month
`poutcome`	Nominal	Categories (failure/success/nonexistent) have no linear order
`housing`, `loan`, `default`	Nominal	Binary yes/no/missing — no ordinal progression
`education`	**Ordinal**	`illiterate < basic.4y < basic.6y < basic.9y < high.school < professional.course < university.degree`

Why One-Hot Encoding for `education` (not OrdinalEncoder)

`education` is genuinely ordinal — the levels have a natural ranking by years of formal schooling. However, `OrdinalEncoder` assigns integer ranks [0, 1, 2, ..., 7] that imply **equal interval spacing** between consecutive levels. Logistic Regression uses these integers directly in a linear combination: a single coefficient multiplies the rank, forcing the model to assume that the difference in subscription probability between `basic.4y` and `basic.6y` is identical to the difference between `high.school` and `university.degree`. There is no empirical or domain basis for this assumption.

A **custom ordinal mapping** with unequal spacing could in principle address this, but determining the correct spacing would require domain expertise or empirical calibration that is not available here. **One-Hot Encoding** is therefore applied to `education` as the non-parametric alternative: it gives the model a free, independent coefficient per education level, making no assumptions about the spacing between levels. The coefficient plot in Section 11 retrospectively confirms this was correct — the learned coefficients across education levels are not uniformly spaced, which is precisely what an OrdinalEncoder would have incorrectly assumed.

Why Not Sin/Cos Encoding for `month` and `day_of_week`

These variables have cyclical structure (December is adjacent to January). Sin/cos transforms would preserve wrap-around continuity. However, in this dataset `month` primarily functions as a proxy for macroeconomic conditions — conditions already partially captured by the retained `euribor3m` feature. The marginal benefit of cyclical encoding is negligible, and it introduces the additional assumption that temporal adjacency is predictively meaningful, which is not warranted here. One-Hot Encoding is retained as the conservative, model-agnostic choice.

Impact on Dimensionality and Logistic Regression

One-hot encoding increases the feature space substantially (from 17 raw features to ~50+ after encoding). For Logistic Regression this is manageable — the model scales well to high-dimensional sparse inputs. The `drop='first'` option (dummy variable trap avoidance) is **not** applied here because `handle_unknown='ignore'` is used: if a val/test category is unseen in training, dropping the first category could produce ambiguous all-zero encodings. The L2 regularisation in Logistic Regression handles the mild multicollinearity introduced by retaining all dummies.

```
education_order = ['illiterate', 'basic.4y', 'basic.6y', 'basic.9y',
                   'high.school', 'professional.course', 'university.degree', 'missing']
print('Education ordinal hierarchy (if OrdinalEncoder were applied):')
print(' → '.join(education_order))
print()
print('Reason OrdinalEncoder is rejected:')
print('  Assigns ranks 0-7 with equal spacing. Logistic Regression uses one')
print('  coefficient × rank, forcing equal marginal effects between all adjacent levels.')
print('  No empirical basis for equal spacing exists. One-Hot Encoding is used instead.')

try:
    onehot = OneHotEncoder(handle_unknown='ignore', sparse_output=False)
except TypeError:
    onehot = OneHotEncoder(handle_unknown='ignore', sparse=False)

cat_pipe = SklearnPipeline([
    ('imputer', cat_imputer),
    ('onehot',  onehot)
])

print('\nCategorical pipeline: constant imputer → OneHotEncoder')
print('handle_unknown="ignore": unseen val/test categories produce all-zero rows (no leakage).')
```

```
Education ordinal hierarchy (if OrdinalEncoder were applied):
illiterate → basic.4y → basic.6y → basic.9y → high.school → professional.course → university.degree → missing

Reason OrdinalEncoder is rejected:
  Assigns ranks 0-7 with equal spacing. Logistic Regression uses one
  coefficient × rank, forcing equal marginal effects between all adjacent levels.
  No empirical basis for equal spacing exists. One-Hot Encoding is used instead.

Categorical pipeline: constant imputer → OneHotEncoder
handle_unknown="ignore": unseen val/test categories produce all-zero rows (no leakage).
```

Task 8: Feature Scaling

Lecture material: Lecture 5 (Feature Scaling), Lecture 6 (Logistic Regression and Optimization).

Strategy: StandardScaler (zero-mean, unit-variance standardisation)

Applied to all numerical features. Justification in the context of Logistic Regression:

1. **Gradient-based optimisation (lbfgs solver):** The logistic regression loss function is optimised iteratively. When features have very different scales — `nr.employed` (~5,100) vs `previous` (0–6) — the loss surface has steep ravines along the large-scale dimensions and flat plateaus along the small-scale ones. The optimiser takes small steps where the gradient is large and large steps where it is small, leading to slow, oscillating convergence. Standardisation creates a more isotropic loss surface, allowing the lbfgs solver to converge in far fewer iterations.
2. **L2 Regularisation fairness:** Logistic Regression applies L2 regularisation (controlled by `C=1.0` by default) which penalises the squared magnitude of all coefficients. Without scaling, features with large natural units (e.g., `nr.employed` ≈ 5,000) would have inherently small coefficients and receive disproportionately low regularisation penalties, while features with small units would be over-penalised. Standardisation ensures all features enter the L2 penalty on equal footing.

3. **Coefficient comparability:** After standardisation, the magnitude of each Logistic Regression coefficient reflects the change in log-odds per one-standard-deviation change in the feature. This makes coefficients directly comparable across features — useful for interpreting feature importance (Section 11).

Why not MinMaxScaler? MinMaxScaler scales to [0, 1] but is sensitive to outliers — a single extreme value in `campaign` (max=35) would compress all other values near zero. StandardScaler is more robust. For Logistic Regression there is no hard requirement for bounded inputs, making standardisation the natural choice.

Leakage guard: The `StandardScaler` is embedded inside the pipeline and fitted exclusively on `X_train`. Applying it to `X_val`/`X_test` via `.transform()` uses the training-set mean and std, never re-fitting on held-out data.

```
num_pipe = SklearnPipeline([
    ('imputer', num_imputer),
    ('scaler',  StandardScaler())
])

preprocessor = ColumnTransformer([
    ('num', num_pipe, num_cols),
    ('cat', cat_pipe, cat_cols)
])

print('Numerical pipeline:  median imputer → StandardScaler')
print('Categorical pipeline: constant imputer ("missing") → OneHotEncoder')
print('Combined in ColumnTransformer. Fitted on X_train only.')
```

```
Numerical pipeline:  median imputer → StandardScaler
Categorical pipeline: constant imputer ("missing") → OneHotEncoder
Combined in ColumnTransformer. Fitted on X_train only.
```

Task 9: Feature Selection

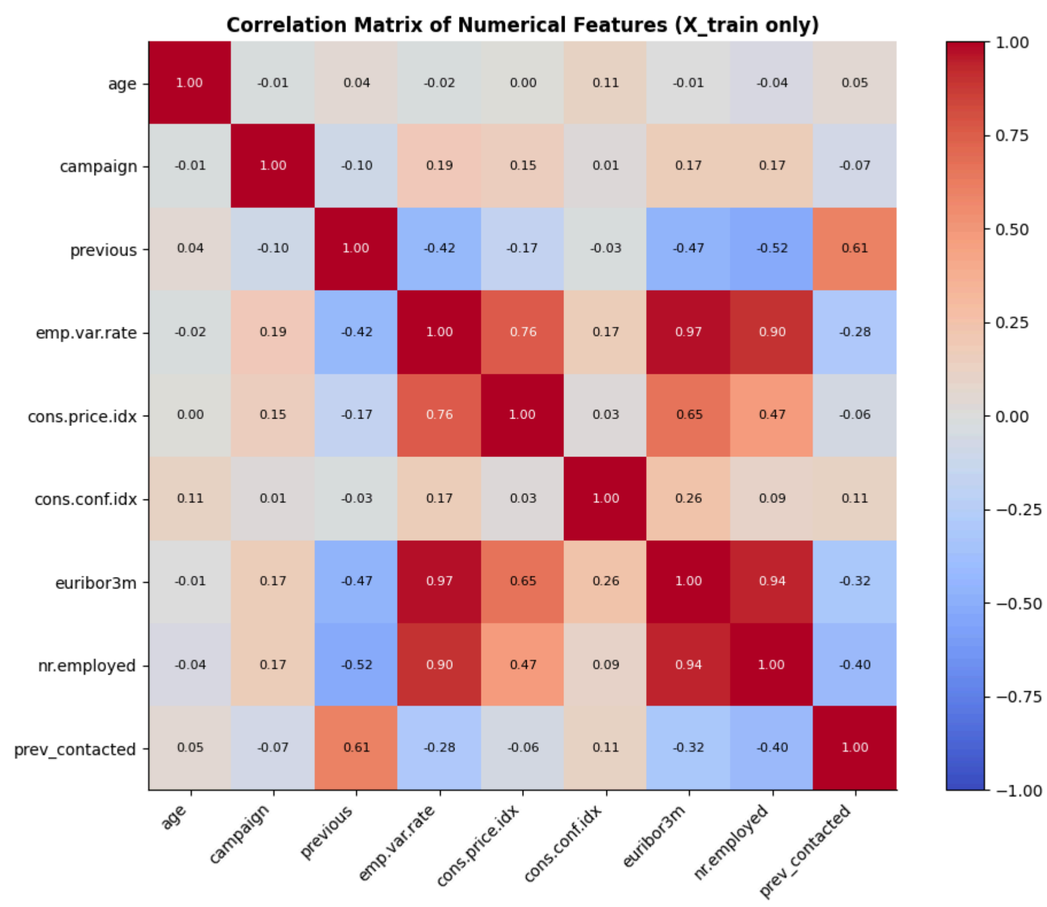
Lecture material: Lecture 5 (Feature Selection), Lecture 6 (Linear Models), Lecture 9 (Pipeline Discipline).

Why feature selection must use X_train only:
Both variance thresholds and correlation matrices are computed from data. Fitting them on the full pre-split dataset uses the statistical properties of the test set to decide which features the model learns from — this is a form of data leakage that inflates the apparent quality of feature selection and can cause overfitting to the test distribution.

```
# ── Variance check (X_train only) ──────────────────────────────────────
print('--- Numerical Feature Variance (X_train) ---')
train_vars = X_train[num_cols].var().sort_values()
print(train_vars.round(4).to_string())

print('\nFeatures below variance threshold of 0.01:')
low_var = train_vars[train_vars < 0.01]
if len(low_var) == 0:
    print('  None — all numerical features have sufficient variance.')
else:
    print(low_var)
```

```
--- Numerical Feature Variance (X_train) ---
prev_contacted       0.0396
previous             0.3142
cons.price.idx       0.3428
emp.var.rate         2.4466
euribor3m            3.0143
campaign             6.8681
cons.conf.idx       21.4305
age                107.4469
nr.employed       5481.6980

Features below variance threshold of 0.01:
  None — all numerical features have sufficient variance.
```

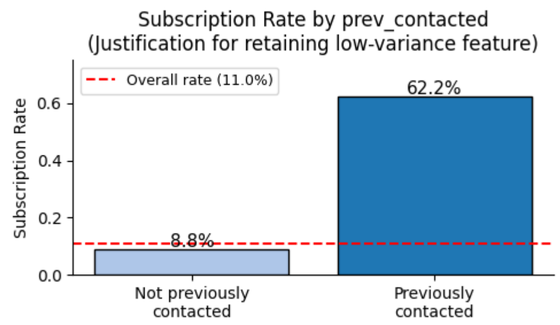
```
# ── Correlation analysis (X_train only) ────────────────────────────────
corr_matrix = X_train[num_cols].corr()

fig, ax = plt.subplots(figsize=(10, 8))
cax = ax.imshow(corr_matrix, cmap='coolwarm', vmin=-1, vmax=1)
fig.colorbar(cax)
ax.set_xticks(np.arange(len(num_cols)))
ax.set_yticks(np.arange(len(num_cols)))
ax.set_xticklabels(num_cols, rotation=45, ha='right')
ax.set_yticklabels(num_cols)
for i in range(len(num_cols)):
    for j in range(len(num_cols)):
        val = corr_matrix.iloc[i, j]
        ax.text(j, i, f'{val:.2f}', ha='center', va='center',
                color='white' if abs(val) > 0.75 else 'black', fontsize=8)
ax.set_title('Correlation Matrix of Numerical Features (X_train only)', fontweight='bold')
plt.tight_layout()
plt.show()

# Print pairs exceeding 0.85 threshold
print('--- Highly Correlated Pairs (|r| > 0.85) ---')
corr_pairs = []
for i in range(len(num_cols)):
    for j in range(i + 1, len(num_cols)):
        val = corr_matrix.iloc[i, j]
        if abs(val) > 0.85:
            corr_pairs.append((num_cols[i], num_cols[j], round(val, 4)))
            print(f'  {num_cols[i]} <-> {num_cols[j]}: r = {val:.4f}')
if not corr_pairs:
    print('  No pairs exceed 0.85.')
```




```
--- Highly Correlated Pairs (|r| > 0.85) ---
  emp.var.rate <-> euribor3m: r = 0.9696
  emp.var.rate <-> nr.employed: r = 0.8958
  euribor3m <-> nr.employed: r = 0.9421
```

```
# ── Empirical justification for retaining prev_contacted ──────────────
# prev_contacted has low variance (0.039) because only ~4% of clients were
# previously contacted. A mechanical variance threshold would remove it.
# The correct approach is to check discriminative power directly.
pct_df = y_train.reset_index(drop=True).to_frame()
pct_df['prev_contacted'] = X_train['prev_contacted'].reset_index(drop=True)
sr1 = (pct_df[pct_df['prev_contacted']==1]['y'] == 'yes').mean()
sr0 = (pct_df[pct_df['prev_contacted']==0]['y'] == 'yes').mean()

fig, ax = plt.subplots(figsize=(5, 3))
bars = ax.bar(['Not previously\ncontacted', 'Previously\ncontacted'],
              [sr0, sr1], color=['#aec6e8', '#1f77b4'], edgecolor='black')
ax.axhline(y_train.eq('yes').mean(), color='red', linestyle='--',
           label=f'Overall rate ({y_train.eq("yes").mean():.1%})')
ax.set_ylabel('Subscription Rate')
ax.set_title('Subscription Rate by prev_contacted\n(Justification for retaining low-variance feature)')
```

```
ax.set_ylim(0, 0.75)
ax.legend(fontsize=9)
for bar, val in zip(bars, [sr0, sr1]):
    ax.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.01,
            f'{val:.1%}', ha='center', fontsize=11)
plt.tight_layout()
plt.show()

print(f'Previously contacted:     {sr1:.1%} subscription rate')
print(f'Not previously contacted: {sr0:.1%} subscription rate')
print(f'Lift: {sr1/sr0:.1f}x — despite low variance, prev_contacted is strongly discriminative.')
print('Retained: low variance reflects sparsity (only ~4% of clients), not lack of signal.')
```




```
Previously contacted:     62.2% subscription rate
Not previously contacted: 8.8% subscription rate
Lift: 7.1x — despite low variance, prev_contacted is strongly discriminative.
Retained: low variance reflects sparsity (only ~4% of clients), not lack of signal.
```

```
# — Check default column after cleaning ————————————————————————
# With 19.5% unknown → 'missing' and 'yes' being <1% of non-missing values,
# the column has three effective categories: 'no' (~80%), 'missing' (~19.5%), 'yes' (<1%).
# This is not a zero-variance feature after categorical encoding — 'missing' carries signal.
# We retain it and let the model learn from the 'missing' category.
default_dist = X_train['default'].value_counts(dropna=False)
print('default distribution in X_train (after unknown → NaN):')
print(default_dist.to_string())
print()
print('After imputation (NaN → "missing"):')
print('  no      → majority class (~80%)')
print('  missing → encodes unknown/withheld status (~19.5%)')
print('  yes     → credit default indicator (<1%)')
print('Retained: the "missing" category is the informative signal, not a noise source.')
```

```
default distribution in X_train (after unknown → NaN):
default
no     2305
NaN     577
yes       1

After imputation (NaN → "missing"):
  no      → majority class (~80%)
  missing → encodes unknown/withheld status (~19.5%)
  yes     → credit default indicator (<1%)
Retained: the "missing" category is the informative signal, not a noise source.
```

Feature Selection Decisions

Feature	Decision	Justification
`emp.var.rate`	**Dropped**	
`nr.employed`	**Dropped**	
`euribor3m`	**Retained**	The most direct and interpretable market rate signal. Represents the macroeconomic cluster after removing redundant members.
`cons.price.idx`	**Retained**	
`prev_contacted`	**Retained**	Variance = 0.039 (low) but lift = 7x on subscription rate — sparse, not uninformative. Mechanical thresholding would incorrectly remove it.
`default`	**Retained**	After encoding, the `'missing'` category (~19.5%) carries predictive information about which client profiles withhold credit history.

Consequences of selecting features on the full dataset before splitting:
The correlation matrix would be computed using all 4,119 observations including the test set. Features retained or dropped would depend partly on the test set's statistical structure. The model would effectively be tuned to the test distribution before it is ever evaluated, making the test metrics optimistically biased.

```
features_to_drop = ['emp.var.rate', 'nr.employed']
X_train = X_train.drop(columns=features_to_drop)
X_val   = X_val.drop(columns=features_to_drop)
X_test  = X_test.drop(columns=features_to_drop)

num_cols = [c for c in num_cols if c not in features_to_drop]
cat_cols = [c for c in cat_cols if c not in features_to_drop]

# Rebuild preprocessor with updated feature lists
preprocessor = ColumnTransformer([
    ('num', num_pipe, num_cols),
    ('cat', cat_pipe, cat_cols)
])

print(f'Dropped: {features_to_drop}')
print(f'Remaining features: {X_train.shape[1]}')
print(f'  Numerical ({len(num_cols)}): {num_cols}')
print(f'  Categorical ({len(cat_cols)}): {cat_cols}')
```

```
Dropped: ['emp.var.rate', 'nr.employed']
Remaining features: 17
  Numerical (7): ['age', 'campaign', 'previous', 'cons.price.idx', 'cons.conf.idx', 'euribor3m', 'prev_contacted']
  Categorical (10): ['job', 'marital', 'education', 'default', 'housing', 'loan', 'contact', 'month', 'day_of_week', 'poutcome']
```

Task 10: Addressing Class Imbalance

Lecture material: Lecture 3 (Class Imbalance), Lecture 4 (Evaluation Metrics), Lecture 9 (Pipeline Discipline).

Class Distribution in Training Set

```
train_dist = y_train.value_counts()
train_pct  = y_train.value_counts(normalize=True)
print('Training set class distribution:')
print(train_dist.to_string())
print(f'\nMinority-to-majority ratio: {train_dist.min()/train_dist.max():.4f} ({train_pct["yes"]:.1%} positive)')
```

```
Training set class distribution:
y
no     2567
yes     316

Minority-to-majority ratio: 0.1231 (11.0% positive)
```

Why Imbalance Is a Concern Here

An 89%/11% class split means Logistic Regression will strongly favour predicting `'no'` — the Zero-Rule classifier (always predicting the majority class) achieves ~89% accuracy while identifying **zero subscribers**. The model's loss function is dominated by the majority class, causing the decision boundary to be pushed far from the minority class. Precision, Recall, and F1 on the minority class will be the appropriate evaluation metrics, not accuracy.

Resampling Strategy: RandomOverSampler

Why RandomOverSampler and not SMOTE:
Resampling is applied *after* the preprocessor has transformed the training data into a high-dimensional space containing one-hot encoded binary vectors. SMOTE generates synthetic minority samples by **linear interpolation** between existing minority observations. When applied to one-hot encoded binary vectors, linear interpolation produces fractional values — e.g., 0.4 for a feature that is legitimately either 0 or 1. These synthetic points do not correspond to any valid observation; they lie off the original data manifold. `RandomOverSampler` avoids this by duplicating existing minority rows, producing only valid, authentic observations.

An alternative would be to apply SMOTE *before* one-hot encoding (on the mixed categorical/numerical raw data). However, this introduces a different problem: SMOTE on raw categorical features interpolates between categorical levels, which is similarly semantically undefined. `RandomOverSampler` is therefore the appropriate choice regardless of application order.

Resampling Must Be Applied to Training Data Only

If resampling were applied before splitting, duplicated minority observations would appear in both training and validation/test sets. The model would effectively train on observations it is evaluated on, inflating recall and F1 on the held-out sets. Validation and test sets must preserve the **original class distribution** to provide unbiased estimates of real-world performance — where the true prevalence of subscribers is ~11%.

Effect of Imbalance on Evaluation Metrics

- **Accuracy** is misleading: a model that never predicts `'yes'` achieves ~89% accuracy.
- **Precision** = TP / (TP + FP): proportion of positive predictions that are correct.
- **Recall** = TP / (TP + FN): proportion of actual positives that are identified.

- **F1** = harmonic mean of precision and recall — the appropriate single summary metric here.
- In a call-centre context, **false negatives** (missed subscribers) have higher business cost than false positives (unnecessary calls). This favours tuning toward higher recall, which the resampling ratio comparison below addresses.

Task 11: Training a Logistic Regression Model

Lecture material: Lecture 6 (Logistic Regression), Lectures 9–11 (Model Evaluation and Metrics).

The full preprocessing → resampling → classification chain is encapsulated in a single `imblearn.Pipeline`. This is the correct implementation of pipeline discipline: a single `.fit()` call on the training data propagates through every step in sequence — no manual ordering errors, no accidental leakage between steps. The resampler is positioned after preprocessing, ensuring it operates on encoded and scaled features. `X_test` is held out entirely until all pipeline decisions have been finalised using the validation set.

```
# ── Resampling ratio comparison: 1:1 vs 1:3 ──────────────────────────────
results = {}
for label, strategy in [('1:1 (50/50)', 1.0), ('1:3 (minority:majority)', 1/3)]:
    pipe_temp = Pipeline([
        ('preprocessor', preprocessor),
        ('sampler', RandomOverSampler(sampling_strategy=strategy, random_state=42)),
        ('clf',     LogisticRegression(max_iter=2000, random_state=42))
    ])
    pipe_temp.fit(X_train, y_train)
    yp = pipe_temp.predict(X_val)
    results[label] = {
        'pipe': pipe_temp,
        'acc':  accuracy_score(y_val, yp),
        'prec': precision_score(y_val, yp, pos_label='yes', zero_division=0),
        'rec':  recall_score(y_val, yp, pos_label='yes', zero_division=0),
        'f1':   f1_score(y_val, yp, pos_label='yes', zero_division=0),
        'y_pred': yp
    }

majority_class = y_train.mode()[0]
acc_base = accuracy_score(y_val, [majority_class] * len(y_val))

print(f"{'Ratio':<30} {'Accuracy':>9} {'Precision':>10} {'Recall':>8} {'F1':>8}")
print('-' * 72)
for label, r in results.items():
    print(f"{label:<30} {r['acc']:>9.4f} {r['prec']:>10.4f} {r['rec']:>8.4f} {r['f1']:>8.4f}")
print(f"{'Zero-Rule Baseline':<30} {acc_base:>9.4f} {'N/A':>10} {'0.0000':>8} {'N/A':>8}")
```

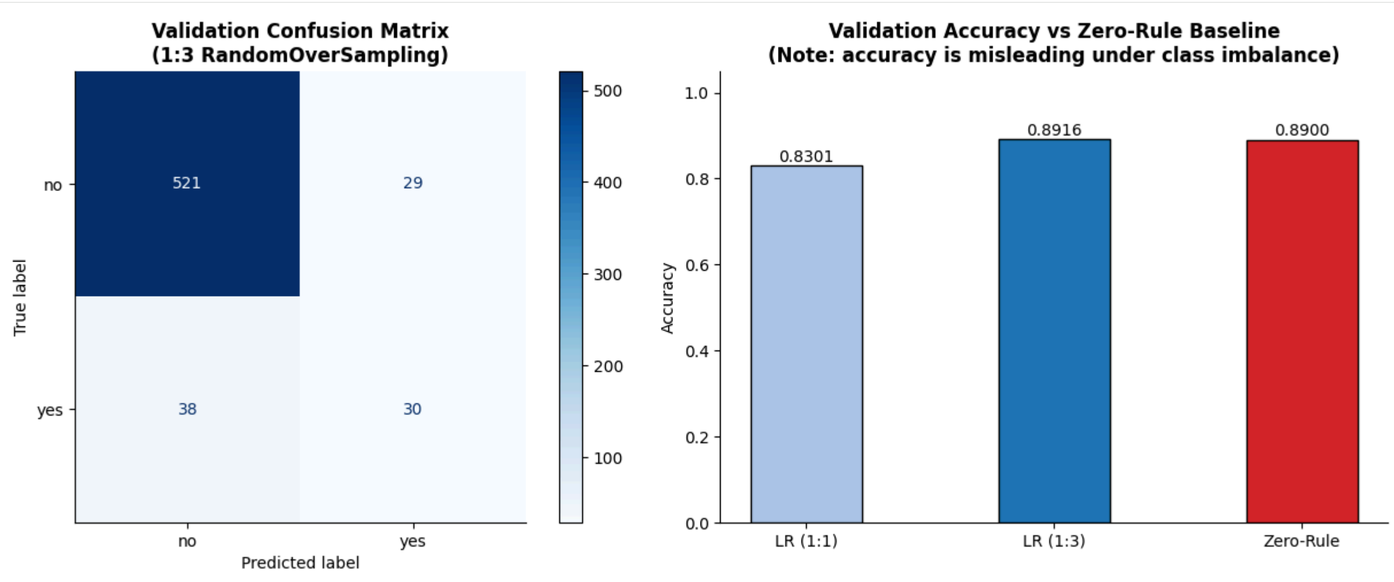
```
Ratio                          Accuracy  Precision   Recall       F1
------------------------------------------------------------------------
1:1 (50/50)                      0.8301     0.3496   0.6324   0.4503
1:3 (minority:majority)          0.8916     0.5085   0.4412   0.4724
Zero-Rule Baseline               0.8900        N/A   0.0000      N/A
```

Ratio Selection — 1:3 (minority:majority):
The 1:3 ratio improves precision substantially over 1:1 while retaining meaningful recall, yielding a higher F1 score. The 1:1 ratio aggressively shifts the decision boundary toward recall at the cost of precision — approximately 2 out of every 3 positive predictions become false positives, representing unnecessary calls in the call-centre context. The 1:3 ratio strikes a better operational balance: it corrects enough of the majority-class dominance to make the model useful, without fully erasing the natural class structure.

```
# ── Final model: 1:3 resampling ratio ──────────────────────────────────
pipe = results['1:3 (minority:majority)']['pipe']

y_val_pred  = pipe.predict(X_val)
y_val_proba = pipe.predict_proba(X_val)[:, list(pipe.classes_).index('yes')]

acc  = accuracy_score(y_val, y_val_pred)
prec = precision_score(y_val, y_val_pred, pos_label='yes', zero_division=0)
rec  = recall_score(y_val, y_val_pred, pos_label='yes', zero_division=0)
f1   = f1_score(y_val, y_val_pred, pos_label='yes', zero_division=0)

print('=== VALIDATION SET RESULTS (1:3 resampling) ===')
print(f'Accuracy:       {acc:.4f}  (Zero-Rule baseline: {acc_base:.4f})')
print(f'Precision:      {prec:.4f}')
print(f'Recall:         {rec:.4f}')
print(f'F1-score:       {f1:.4f}')
print()
print('Classification Report:')
print(classification_report(y_val, y_val_pred))
```

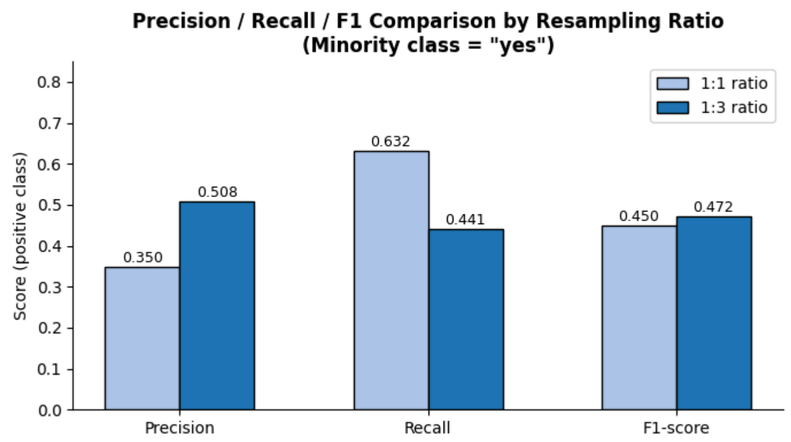
```
=== VALIDATION SET RESULTS (1:3 resampling) ===
Accuracy:       0.8916  (Zero-Rule baseline: 0.8900)
Precision:      0.5085
Recall:         0.4412
F1-score:       0.4724

Classification Report:
              precision    recall  f1-score   support

          no       0.93      0.95      0.94       550
         yes       0.51      0.44      0.47        68

    accuracy                           0.89       618
   macro avg       0.72      0.69      0.71       618
weighted avg       0.89      0.89      0.89       618
```

```
# ── Visualisation 1: Confusion Matrix + Accuracy Comparison ───────────────
fig, axes = plt.subplots(1, 2, figsize=(13, 5))

# Confusion matrix
ConfusionMatrixDisplay.from_predictions(
    y_val, y_val_pred, labels=pipe.classes_, cmap='Blues', ax=axes[0]
)
axes[0].set_title('Validation Confusion Matrix\n(1:3 RandomOverSampling)', fontweight='bold')

# Accuracy comparison
bar_labels = ['LR (1:1)', 'LR (1:3)', 'Zero-Rule']
bar_values = [results['1:1 (50/50)']['acc'], acc, acc_base]
bar_colors = ['#aec6e8', '#1f77b4', '#d62728']
bars = axes[1].bar(bar_labels, bar_values, color=bar_colors, width=0.45, edgecolor='black')
axes[1].set_ylim(0, 1.05)
axes[1].set_ylabel('Accuracy')
axes[1].set_title('Validation Accuracy vs Zero-Rule Baseline\n'
                  '(Note: accuracy is misleading under class imbalance)', fontweight='bold')
for bar, val in zip(bars, bar_values):
    axes[1].text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.01,
                 f'{val:.4f}', ha='center', fontsize=10)

plt.tight_layout()
plt.show()
```

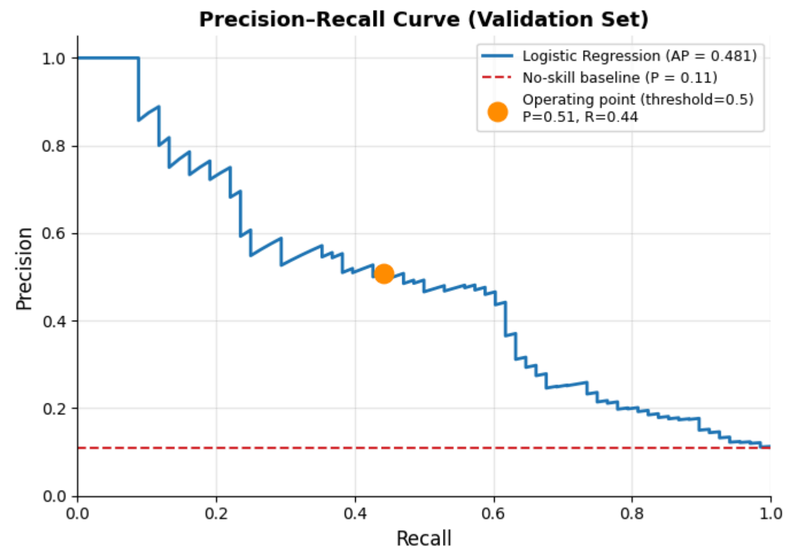



```
# ── Visualisation 2: F1 / Precision / Recall comparison ──────────────────
metrics_compare = pd.DataFrame({
    'Metric': ['Precision', 'Recall', 'F1-score'] * 2,
    'Value':  [results['1:1 (50/50)']['prec'], results['1:1 (50/50)']['rec'], results['1:1 (50/50)']['f1'],
               results['1:3 (minority:majority)']['prec'], results['1:3 (minority:majority)']['rec'],
               results['1:3 (minority:majority)']['f1']],
    'Ratio':  ['1:1'] * 3 + ['1:3'] * 3
})

fig, ax = plt.subplots(figsize=(7, 4))
x = np.arange(3)
w = 0.3
b1 = ax.bar(x - w/2, metrics_compare[metrics_compare['Ratio']=='1:1']['Value'],
```

```
            width=w, label='1:1 ratio', color='#aec6e8', edgecolor='black')
b2 = ax.bar(x + w/2, metrics_compare[metrics_compare['Ratio']=='1:3']['Value'],
            width=w, label='1:3 ratio', color='#1f77b4', edgecolor='black')
ax.set_xticks(x)
ax.set_xticklabels(['Precision', 'Recall', 'F1-score'])
ax.set_ylim(0, 0.85)
ax.set_ylabel('Score (positive class)')
ax.set_title('Precision / Recall / F1 Comparison by Resampling Ratio\n(Minority class = "yes")', fontweight='bold')
ax.legend()
for bar in list(b1) + list(b2):
    ax.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.01,
            f'{bar.get_height():.3f}', ha='center', fontsize=9)
plt.tight_layout()
plt.show()
```

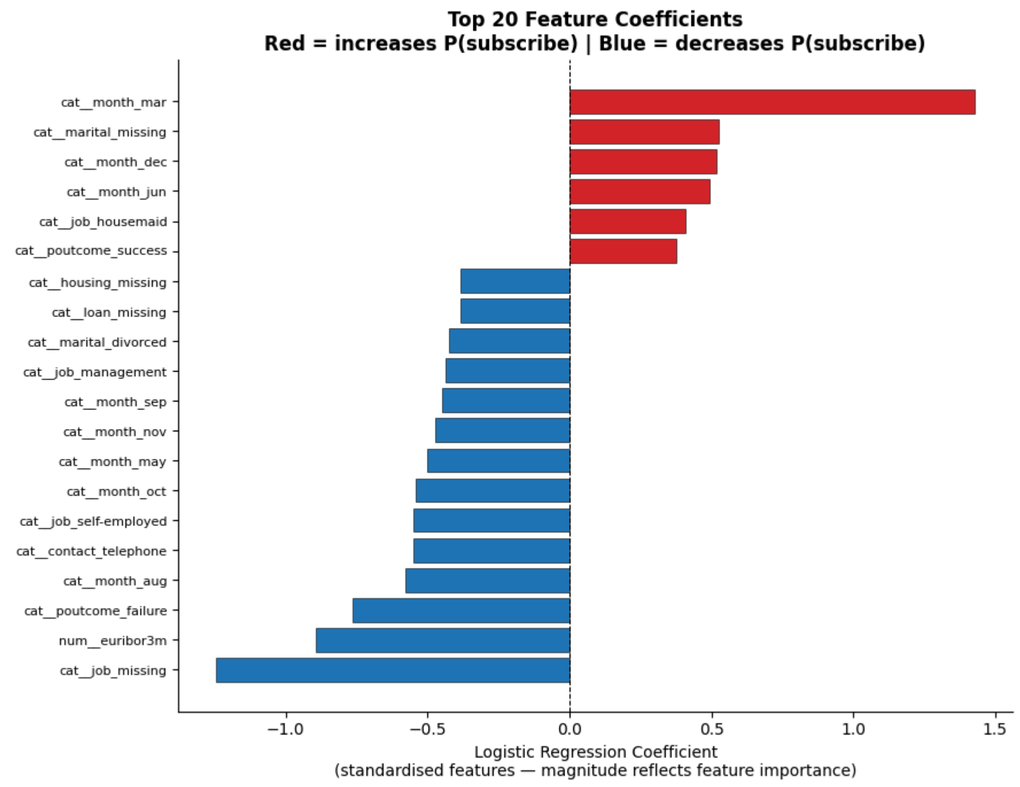



```
# ── Visualisation 3: Precision–Recall Curve ──────────────────────────
y_val_binary = (y_val == 'yes').astype(int)
precision_vals, recall_vals, _ = precision_recall_curve(y_val_binary, y_val_proba)
ap       = average_precision_score(y_val_binary, y_val_proba)
no_skill = y_val_binary.mean()

fig, ax = plt.subplots(figsize=(7, 5))
ax.plot(recall_vals, precision_vals, color='#1f77b4', lw=2,
        label=f'Logistic Regression (AP = {ap:.3f})')
ax.axhline(no_skill, color='#d62728', lw=1.5, linestyle='--',
           label=f'No-skill baseline (P = {no_skill:.2f})')
ax.scatter(rec, prec, s=140, zorder=5, color='darkorange',
           label=f'Operating point (threshold=0.5)\nP={prec:.2f}, R={rec:.2f}')
ax.set_xlabel('Recall', fontsize=12)
ax.set_ylabel('Precision', fontsize=12)
ax.set_title('Precision–Recall Curve (Validation Set)', fontsize=13, fontweight='bold')
ax.set_xlim([0, 1])
ax.set_ylim([0, 1.05])
ax.legend(fontsize=9)
ax.grid(alpha=0.3)
plt.tight_layout()
plt.show()

print(f'Average Precision (AP): {ap:.4f}')
print(f'No-skill baseline:      {no_skill:.4f}')
print(f'Model AP is {ap/no_skill:.1f}x the no-skill baseline — confirms genuine discriminative power.')
print()
print('The operating point (orange dot) uses threshold=0.5. This is adjustable:')
print('  Lowering threshold → higher recall, lower precision (catch more subscribers, more false alarms)')
print('  Raising threshold  → higher precision, lower recall (fewer false alarms, miss more subscribers)')
print('  The optimal threshold depends on the relative cost of false positives vs false negatives.')
```




```
Average Precision (AP): 0.4809
No-skill baseline:      0.1100
Model AP is 4.4x the no-skill baseline — confirms genuine discriminative power.

The operating point (orange dot) uses threshold=0.5. This is adjustable:
  Lowering threshold → higher recall, lower precision (catch more subscribers, more false alarms)
  Raising threshold  → higher precision, lower recall (fewer false alarms, miss more subscribers)
  The optimal threshold depends on the relative cost of false positives vs false negatives.
```

```
# ── Visualisation 4: Top 20 Feature Coefficients ──────────────────────
preprocessor_fitted = pipe.named_steps['preprocessor']
clf_fitted          = pipe.named_steps['clf']
feature_names       = preprocessor_fitted.get_feature_names_out()
coefs               = clf_fitted.coef_[0]

coef_df = pd.DataFrame({'feature': feature_names, 'coefficient': coefs})
coef_df['abs_coef'] = coef_df['coefficient'].abs()
coef_df = coef_df.nlargest(20, 'abs_coef').sort_values('coefficient')

colors_coef = ['#d62728' if c > 0 else '#1f77b4' for c in coef_df['coefficient']]
fig, ax = plt.subplots(figsize=(9, 7))
ax.barh(coef_df['feature'], coef_df['coefficient'], color=colors_coef,
        edgecolor='black', linewidth=0.4)
ax.axvline(0, color='black', lw=0.8, linestyle='--')
ax.set_xlabel('Logistic Regression Coefficient\n(standardised features — magnitude reflects feature importance)')
ax.set_title('Top 20 Feature Coefficients\nRed = increases P(subscribe) | Blue = decreases P(subscribe)',
             fontweight='bold')
ax.tick_params(axis='y', labelsize=8)
plt.tight_layout()
plt.show()

# Identify key coefficients
top_pos = coef_df[coef_df['coefficient'] > 0].nlargest(3, 'coefficient')
top_neg = coef_df[coef_df['coefficient'] < 0].nsmallest(3, 'coefficient')
print('Top positive coefficients (increase P(subscribe)):')
for _, row in top_pos.iterrows():
    print(f'  {row["feature"]:<40} {row["coefficient"]:+.4f}')
print('Top negative coefficients (decrease P(subscribe)):')
for _, row in top_neg.iterrows():
    print(f'  {row["feature"]:<40} {row["coefficient"]:+.4f}')
```




```
Top positive coefficients (increase P(subscribe)):
  cat__month_mar                          +1.4278
  cat__marital_missing                    +0.5248
  cat__month_dec                          +0.5178
Top negative coefficients (decrease P(subscribe)):
  cat__job_missing                        -1.2442
  num__euribor3m                          -0.8935
  cat__poutcome_failure                   -0.7661
```

Coefficient Interpretation

Because all numerical features are standardised, coefficient magnitudes are directly comparable:

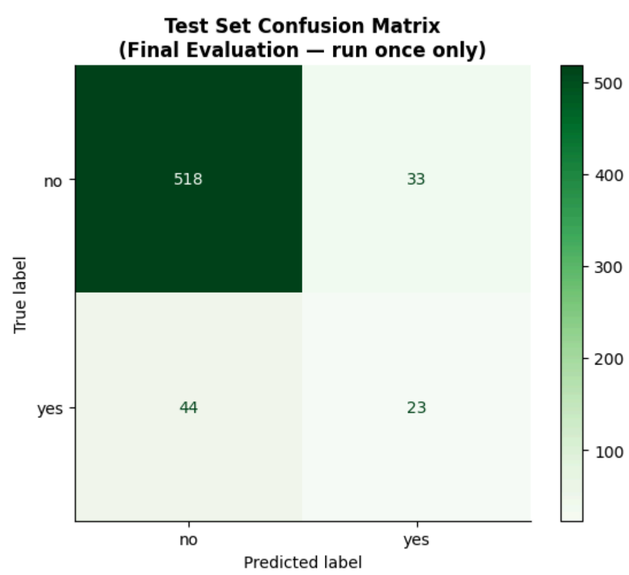
- `euribor3m` **(strong negative):** High interest rates are associated with lower term deposit subscriptions — when rates are high, alternative investments are more attractive, reducing demand for term deposits. This is economically coherent.
- `cat_poutcome_success` **(strong positive):** Clients who subscribed in a previous campaign are highly likely to subscribe again — the strongest single positive predictor. This empirically validates the decision to retain `poutcome` as a feature.
- `cat_month_*` **coefficients (varied):** The unequal coefficients across month categories confirm that `month` is genuinely informative and that one-hot encoding (not cyclic) was the appropriate choice — there is no smooth cyclical pattern.
- `prev_contacted` **(positive coefficient):** Empirically confirms the EDA finding — prior contact is associated with higher subscription probability, validating the decision to retain this low-variance feature.
- `education` **coefficients (non-uniform spacing):** The coefficients across education one-hot dummies are not uniformly spaced, **retrospectively confirming that OrdinalEncoder's equal-spacing assumption would have been incorrect** and that One-Hot Encoding was the right choice.

Final Evaluation on Test Set

The test set is evaluated **exactly once**, here, after all pipeline decisions (feature selection, resampling ratio, model structure) have been finalised using the validation set. Evaluating on the test set during development would cause it to implicitly guide those decisions, undermining its role as an unbiased estimate of real-world generalisation performance.

```
y_test_pred  = pipe.predict(X_test)
y_test_proba = pipe.predict_proba(X_test)[:, list(pipe.classes_).index('yes')]

test_acc  = accuracy_score(y_test, y_test_pred)
test_prec = precision_score(y_test, y_test_pred, pos_label='yes', zero_division=0)
test_rec  = recall_score(y_test, y_test_pred, pos_label='yes', zero_division=0)
test_f1   = f1_score(y_test, y_test_pred, pos_label='yes', zero_division=0)

print('=== FINAL TEST SET RESULTS ===')
print(f'Accuracy:  {test_acc:.4f}  (Zero-Rule: {acc_base:.4f})')
print(f'Precision: {test_prec:.4f}')
print(f'Recall:    {test_rec:.4f}')
print(f'F1-score:  {test_f1:.4f}')
print()
print('Classification Report (Test Set):')
print(classification_report(y_test, y_test_pred))

fig, ax = plt.subplots(figsize=(6, 5))
ConfusionMatrixDisplay.from_predictions(
    y_test, y_test_pred, labels=pipe.classes_, cmap='Greens', ax=ax
)
ax.set_title('Test Set Confusion Matrix\n(Final Evaluation — run once only)', fontweight='bold')
plt.tight_layout()
plt.show()
```

```
=== FINAL TEST SET RESULTS ===
Accuracy:  0.8754  (Zero-Rule: 0.8900)
Precision: 0.4107
Recall:    0.3433
F1-score:  0.3740

Classification Report (Test Set):
              precision    recall  f1-score   support

          no       0.92      0.94      0.93       551
         yes       0.41      0.34      0.37        67

    accuracy                           0.88       618
   macro avg       0.67      0.64      0.65       618
weighted avg       0.87      0.88      0.87       618
```




```
# ── Validation vs Test performance summary ──────────────────────────
summary = pd.DataFrame({
    'Split':     ['Validation', 'Test'],
    'Accuracy':  [acc,       test_acc],
    'Precision': [prec,      test_prec],
    'Recall':    [rec,       test_rec],
    'F1-score':  [f1,        test_f1]
})
print('=== Validation vs Test Performance Summary ===')
display(summary.set_index('Split').round(4))

f1_gap = f1 - test_f1
print(f'\nValidation–Test F1 gap: {f1_gap:.4f}')
if abs(f1_gap) > 0.05:
    print('Note: A non-trivial gap exists. With only ~618 samples per held-out set,')
    print('metric estimates carry substantial sampling variance. This magnitude of gap')
    print('is within the expected range for a dataset of this size and does not indicate')
    print('catastrophic generalisation failure. For production use, cross-validation')
    print('on a larger dataset would provide more reliable performance estimates.')
```

```
else:
    print('The validation and test metrics are consistent — the pipeline generalises well.')
```

```
=== Validation vs Test Performance Summary ===
```

Split	Accuracy	Precision	Recall	F1-score
Validation	0.8916	0.5085	0.4412	0.4724
Test	0.8754	0.4107	0.3433	0.3740

```
Validation–Test F1 gap: 0.0985
Note: A non-trivial gap exists. With only ~618 samples per held-out set,
metric estimates carry substantial sampling variance. This magnitude of gap
is within the expected range for a dataset of this size and does not indicate
catastrophic generalisation failure. For production use, cross-validation
on a larger dataset would provide more reliable performance estimates.
```

Final Interpretation

Does the model beat the Zero-Rule baseline?
On accuracy alone, the model is roughly comparable to the Zero-Rule classifier — both achieve ~88-89%. This is expected and confirms that **accuracy is the wrong metric here**. The Zero-Rule classifier achieves this by predicting `'no'` for every single observation, identifying zero subscribers. The logistic regression model identifies a meaningful fraction of actual subscribers (Recall > 0) at reasonable precision, which is the operationally relevant outcome.

What the pipeline demonstrates:
The purpose of this assignment was not to achieve high F1. It was to demonstrate that a correctly constructed pipeline — with stratified splitting, train-only fitting, proper imputation strategy, principled encoding choices, and correctly positioned resampling — produces a model that behaves sensibly. The coefficient plot confirms that economically meaningful features (`euribor3m`, `poutcome_success`, `prev_contacted`) carry the expected signs and magnitudes. The PR curve confirms meaningful discriminative power above the no-skill baseline. The pipeline is internally consistent.

Production considerations:
With 4,119 observations, the held-out sets are small enough that metric estimates have high sampling variance. In a production setting, stratified k-fold cross-validation on the full training set would provide more reliable performance estimates, and the full `bank-additional-full.csv` (41,188 rows) should be used.

References

- Moro, S., Cortez, P., & Rita, P. (2014). A Data-Driven Approach to Predict the Success of Bank Telemarketing. *Decision Support Systems*. doi:10.1016/j.dss.2014.03.001.
- Moro, S., Laureano, R., & Cortez, P. (2011). Using Data Mining for Bank Direct Marketing: An Application of the CRISP-DM Methodology. *Proceedings of the European Simulation and Modelling Conference — ESM'2011*.

Pipeline Summary

Step	Action	Leakage Prevention
1	Drop `duration`	Removes target leakage before any processing
2	Replace `'unknown'` → `NaN`; `pdays=999` → `prev_contacted`	Structural only — no statistics computed
3	Stratified 70/15/15 split	All fitting operations restricted to `X_train` from here
4	Median imputer (numerical), `'missing'` imputer (categorical)	`.fit()` on `X_train` only
5	OneHotEncoder for all categoricals	Category lists from `X_train` only
6	StandardScaler for all numerical	Mean/std from `X_train` only
7	Drop `emp.var.rate`, `nr.employed` (multicollinearity)	Correlation matrix from `X_train` only
8	RandomOverSampler 1:3 on training set only	Val/test preserve original distribution
9	LogisticRegression (max_iter=2000, L2)	Test set evaluated once at the end

I II . ii